

→ Phase 2: Integration Testing and Coverage

Objectives

The objective of phase 2 is to apply the following techniques and tools in a practical case study:

- Software integration,
- Integration testing, and
- Measuring code coverage and testability.

In addition, for the modules that are to be developed for this phase, we will keep on practicing test-driven development.

General Description

This phase of the project concerns developing, integrating and integration testing of an application that represents the autonomous parking assistant. The module is implemented by making calls to the methods of the class implemented in Phase 1. In this phase, you are supposed to integrate sensor and actuator classes to the class designed in the Phase 1. Sensors are the same ultrasound sensors that you used in Phase 1, but this time they need to go beyond returning random or fixed values. Actuators are the controllers of the car engine that will be called in the **MoveForward** and **MoveBackward** methods of Phase 1. Please see below for a more detailed description.

The sensors report the actual status of available parking places if they work well or otherwise generates out-of-bound values. You first design the interface of the sensor class and then mock it using Mockito as described below.

Similarly, the actuator receives a call from the class designed in Phase 1 to move the car forward and backward and maintains the current car position so that it does not accept a command to move beyond limits. Again you design the interface of this class and mock it using Mockito.

The sensor data should be mocked such that it represent a street with three parking places of mutually different sizes, one should be not enough for safe parking and the other two enough for parking. Moreover, one of the sensors should get broken halfway in the middle of the scenario (i.e., when the car has reached the middle of the street while moving forward) so that it constantly produces recognizably out-of-bound values. It is assumed that the layout of the street is fixed, that is, no other cars park or unpark at the same time. The layout of the street with the fixed parking places may be hard-coded in the sensor mock class or can be read from a file, and if there is any need, you can define more than one street layout.

Deliverables

There are two main deliverables for this phase:

- a single PDF file documenting the outcome of each and every of the following steps,

- a ZIP file containing the source code of the software implemented in different parts, and

The deliverables are to be submitted in **on Blackboard**.

The deadline for submitting this phase deliverables is October 2, 2026.

Part 1: Architectural Design

First, make an architectural design of the application and organize the relationship among the classes in a class diagram. (see, for example, [IBM Rational's Tutorial on UML Structural Diagrams](https://www.ibm.com/developerworks/rational/library/content/RationalEdge/sep04/bell/) (<https://www.ibm.com/developerworks/rational/library/content/RationalEdge/sep04/bell/>)). Pick an appropriate design pattern for the task.

The module you designed in phase 1 will certainly be part of this architectural design. However, some changes may be necessary as you come to understand the actual working of the module and also its interaction with the outside modules.

Part 2: TDD of New Modules

If you design any new modules in this phase, they have to be designed in a TDD manner and their specification (interface, description, pre- and post-conditions, and test cases) are to be included in the report. You do not need to include extra information about the techniques used to design the test-cases.

Part 3: Integration Testing

Think of 2 scenarios that capture the main functionalities of the application. An example scenario is the following: A car

- Starts at the beginning of the street,
- Moves along the street and scan the available parking places,
- Moves backwards until the most efficient parking place (the smallest available parking where it can still park safely),
- Parks the car,
- Unparks the car and drive to the end of the street.

Implement this scenarios with your mock-ups using [Mockito](http://site.mockito.org/) (<http://site.mockito.org/>), alternatively provide manual mock ups for the required components.

Measure statement and interface coverage of your application using [EclEmma](http://www.eclEmma.org/) (<http://www.eclEmma.org/>) (or similar tools). A somewhat old video on how this is done in Eclipse is here:

If your statement coverage is not 100%, design at least 2 additional scenarios such that each of them will increase at least one of your coverage criteria. Document the scenario's and the reason you think they contribute to more coverage. Implement them in your mock-up and document the resulting additional coverage for each and every scenario.